Multimedia Computing – Assignment 1

Table of Contents

Introduction

This assignment is divided into three sections: **Audio Processing**, **Image Processing**, and **Video Processing**. Each section contains two tasks, which you are required to complete by writing Python scripts. Your task is to process the provided files according to the instructions and submit the results to the eClass platform.

You are expected to submit **six Python scripts** (two for each section) and a **requirements.txt** file that lists all the dependencies required to run your code. Failure to follow submission guidelines or file naming conventions will result in a zero score for that specific task.

Please make sure to read the instructions carefully to avoid errors that could lead to discrepancies in grading.

Audio Processing

Task 1: Waveform Analysis

- Objective: Analyze and visualize the waveform of an audio file.
- Instructions:
  i. Use the provided `original_audio.wav` file in eclass.
  ii. Extract the waveform and plot it using Python libraries (e.g., `matplotlib`).
  iii. Save the plot as `1_audio_waveform.png`.
- Expected Submission:
  A Python script named `1_audio_waveform.py` that generates the waveform plot and saves it as `1_audio_waveform.png`.
- Make sure figure size is (10, 4) inches using `matplotlib.pyplot`

Task 2: Audio Compression

- Objective: Compress the audio file by reducing its sample rate and file size.(when compression_ratio = compressed_size / original_size, compression_ratio < 0.7)
- Instructions:
  i. Compress the provided `original_audio.wav` file.
  ii. Apply a suitable compression technique and reduce the sample rate **twice**.
  iii. Save the compressed file as `2_audio_compression.wav`.
- Expected Submission:
  A Python script named `2_audio_compression.py` that compresses the audio and saves it as `2_audio_compression.wav`.

Important Notes on Grading for Audio Section

In the audio section of this assignment, to avoid discrepancies and ensure that your submission is correctly graded, **you must use the following libraries**(alternative library usage results in 0):

- **NumPy** (`numpy`)
- **Matplotlib** (`matplotlib`)
- **SciPy** (`scipy.io`)

These libraries have been explicitly chosen for their ease of use and compatibility with the grading scripts. Do not use alternative libraries that perform similar tasks, such as `librosa` or `wave`, as this could result in different outputs that will not match the reference files, causing your submission to be graded incorrectly.

Image Processing

In this section, you will be working with an RGB image (`original_image.png`) provided to you in eclass. Your task is to apply the following three image processing techniques and save the results accordingly.

Task 1: Intensity Transformation (Logarithmic Transformation)

Objective:

Apply a logarithmic intensity transformation to enhance the contrast of the grayscale version of the original RGB image.

Instructions:

1. Input:
    - Read the provided RGB image (`original_image.png`).
    - Convert the image from RGB to Grayscale before applying the transformation.
2. Do not forget to normalize it first
3. Transformation:

Apply the logarithmic intensity transformation using the formula:
`I_new = c * log(1 + I_old)`

- Where:
    - `I_new` is the transformed pixel intensity.
    - `I_old` is the original pixel intensity (after converting the image to grayscale).

`c` is a scaling constant calculated using the formula:
`c = 255 / log(1 + max(I_old))`

- Where `max(I_old)` is the maximum pixel intensity in the grayscale image.

4. Output:
    - Save the transformed image as `1_image_log_transform.png`.

Expected Submission:

- A Python script named `1_image_log_transform.py` that:
    - Reads the input RGB image (`original_image.png`).
    - Converts it to grayscale.
    - Applies the logarithmic transformation to enhance contrast.
    - Saves the transformed image as `1_image_log_transform.png`.

Task 2: Feature Extraction (Edge Detection with Sobel Operator)

Objective:

Apply Sobel edge detection to the grayscale version of the original RGB image to detect edges.

Instructions:

1. Input:
    - Read the provided RGB image (`original_image.png`).
    - Convert the image from RGB to Grayscale before applying edge detection.
2. Transformation:
    - Apply the Sobel operator in both the x and y directions to detect edges. The Sobel kernels are as follows:

        Sobel kernel for the x-direction (G_x):

        $$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

        Sobel kernel for the y-direction (G_y):

        $$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

    - After applying the kernels, combine the results of G_x and G_y using the following formula to calculate the gradient magnitude:

        $$G = \sqrt{(G_x^2 + G_y^2)}$$

3. Output:
    - Save the edge-detected image as `2_image_sobel_edges.png`.

Expected Submission:

- A Python script named `2_image_sobel_edges.py` that:
    - Reads the input RGB image (`original_image.png`).
    - Converts it to grayscale.
    - Applies the Sobel operator to detect edges in both the x and y directions.
    - Saves the resulting edge-detected image as `2_image_sobel_edges.png`.

Important Notes for Avoiding Grading Discrepancies:

To ensure that your submission is graded correctly:

1. Use the provided constants and methods: If the instructions specify a constant (like the scaling constant `c`) or a specific transformation matrix, make sure to use it exactly as instructed. Deviating from the specified constants or formulas can lead to different results and incorrect grading.
2. Follow the correct flow: Ensure that you follow the sequence of operations as mentioned, especially when converting the image to grayscale or applying transformations in the correct order.
3. Output formats: Make sure your output images are saved in PNG format with the correct file names as indicated. Incorrect file formats or names will result in zero points for that task.

In the image section of this assignment, to avoid discrepancies and ensure your submission is graded correctly, **you must use the following libraries**(alternative library usage results in 0):

- **Pillow** (`PIL.Image`)
- **NumPy** (`numpy`)
- **OpenCV** (`cv2`)

These libraries are explicitly chosen for their compatibility with the grading scripts. **Do not use alternative libraries** (e.g., `scikit-image`, `imageio`, etc.) for the same tasks, as this could result in outputs that do not match the reference files, leading to incorrect grading.

Video Processing

This section of Assignment 1 focuses on video processing tasks. You are required to complete **three tasks**, each involving a different aspect of video manipulation. The tasks are:

1. Downscaling a video by 50%
2. Blurring a video

Each task must be performed using **Python**, and you are required to submit **two Python scripts** that accomplish these tasks. You will also need to generate the output videos based on the original video provided but not submit them.

Task 1: Downscaling the Video by 50%

Objective: Downscale the resolution of the given video by 50%.

Instructions:

1. **Input**: Use the provided original video named `original_video.mp4`.
2. **Downscaling**:
    - Resize the video frames to **50%** of the original width and height.
    - Use OpenCV's `cv2.resize` function with the **bilinear interpolation method (`cv2.INTER_LINEAR`)** to resize each frame. Do not use other interpolation methods.
    - Calculate the downscaled width and height by dividing the original dimensions by **2**.
3. **Output**: Save the downscaled video as `2_video_downscale.mp4`.

Constraints:

- Maintain the **frame rate (fps)** of the original video.
- **Grading Requirement**: The width, height, and frame rate of your downscaled video will be compared with the reference output. Ensure you are using `cv2.resize` with `cv2.INTER_LINEAR` and that you downscale the video to exactly 50% of its original size.

Task 2: Video Blurring

Objective: Apply a Gaussian blur to the given video.

Instructions:

1. **Input**: Use the provided original video named `original_video.mp4`.
2. **Blurring**:
    - Apply a **Gaussian blur** with a kernel size of `(15, 15)` and standard deviation `0` to each frame of the video.
    - Use OpenCV's `cv2.GaussianBlur` function to apply the blur.
3. **Output**: Save the blurred video as `2_video_blur.mp4`.

Constraints:

- Maintain the **frame rate (fps)**, **width**, and **height** of the original video.
- **Grading Requirement**: The frame rate, width, and height will be compared, as well as the blurring applied to each frame. Ensure you are using a Gaussian blur with the exact kernel size and standard deviation specified (`cv2.GaussianBlur(frame, (15, 15), 0)`).

Submission Guidelines

You are required to submit the following **two Python scripts** and ensure they are correctly named:

1. **1_video_downscale.py**: The script for Task 1 (Downscaling by 50%)
2. **2_video_blur.py**: The script for Task 2 (Blurring)

Additionally, the following **three videos** must be generated but NOT submitted:

1. **1_video_downscale.mp4**: The downscaled output from Task 1
2. **2_video_blur.mp4**: The blurred output from Task 2

Submit the **Python scripts** directly to the eClass system. **Do not zip your files** or place them in folders.

Important Notes on Grading

- Your submission will be graded based on a comparison with reference outputs. The **width**, **height**, **frame rate (fps)**, and **processed frames** (grayscale, downscaling, blurring) will be matched against the reference files.
- **Exact methods and parameters**: You must use the specified methods (such as `cv2.resize`, `cv2.GaussianBlur`, and `cv2.cvtColor`) and parameters to avoid discrepancies in grading. **Different methods or constant values** will likely lead to incorrect grading, even if the output looks correct.
- Ensure the final video dimensions, frame rate, and transformations match the original video (except for the downscaling task).

In this video section of this assignment, **You must use the following library** for all video processing tasks(alternative library usage results in 0):

- **OpenCV** (`cv2`)

Using different libraries for video manipulation may produce different results, and it might complicate running your scripts and grading. Stick to the specified methods.

What to Submit

You are required to submit 6 Python files and 1 `requirements.txt` file. Ensure that you follow the naming conventions strictly:

- 6 Python files (named as shown below) and a requirements.txt file.
    - `1_audio_waveform.py`
      `2_audio_compression.py`
    - `1_image_log_transform.py`
      `2_image_sobel_edges.py`
    - `1_video_downscale.py`
      `2_video_blur.py`
    - `requirements.txt`
- Naming convention: All file names should exactly follow the format mentioned above, including the task numbers. Any deviations will result in zero points for that task.
- Do not zip or place files in folders; upload them directly to eClass.

Important:

- **Do not zip your files**
- **Do not** submit them inside **any folders**.
- Incorrectly named files or creating folders, not following naming protocol will receive **no score** for that task.

Late Submission

- Up to 3 hours late: 30% of the total achievable score will be deducted. - 5
- Next 3 hours: 50% of the total achievable score will be deducted. - 9
- Up to 24 hours: 70% of the total achievable score will be deducted. - 13
- Beyond 24 hours: No submissions will be accepted, and you will receive a score of 0.

Plagiarism

All the submissions will be checked against plagiarism using plagiarism tools.

Any case, even a single case of plagiarism in any of the tasks will result in getting 0 for the entire assignment 1.

Grading

Each task is worth 2 points, making a total of 15 points across the three sections (Audio, Image, and Video).

Not following instructions will result in a penalty.

Section	Task Name	Max Score
Audio	Waveform Analysis	2
	Audio Compression	2
Image	Logarithmic Intensity Transform	2
	Sobel Edge Detection	3
Video	Downscaling to 50%	3
	Video Blurring	3
Total		15